

This comprehensive note outlines the essential steps and technologies required to secure a modern web application, based on the provided sources.

Core Security Principles

- **Never Trust the Client:** This is the foundational rule of web security. All data coming from the browser must be treated as potentially malicious and validated on the server 1-3.
- **Defense in Depth:** Use multiple layers of security (e.g., HTTPS + hashed passwords + rate limiting + RBAC) so that if one layer fails, others remain to protect the system 4.
- **Principle of Least Privilege:** Grant users and systems only the minimum access levels necessary to perform their specific roles 4-6.

Frontend Security: Protecting the Browser

- **Prevent Cross-Site Scripting (XSS):**
- **Validation:** Check if input meets business rules (e.g., is it a valid email format?) 7-9.
- **Sanitization:** Use libraries like **DOMPurify** to clean HTML/Markdown input by removing dangerous scripts while keeping safe formatting 10, 11.
- **Escaping:** Convert special characters (like < to <) at the last moment before rendering to ensure the browser treats them as text, not executable code 12, 13.
- **Safe Rendering:** Avoid dangerous methods like `innerHTML`; instead, use `textContent` or modern frameworks like React that escape text automatically 14, 15.
- **Implement Content Security Policy (CSP):** Use this HTTP header to create a "whitelist" of trusted sources for scripts, styles, and images, which can block malicious scripts even if an XSS vulnerability exists 16, 17.

Transition Security: Protecting Data in Flight

- **Enforce HTTPS Everywhere:** Use **TLS encryption** to protect sensitive data (passwords, tokens) from man-in-the-middle attacks where hackers intercept unencrypted traffic 18-20.
- **Use HSTS (HTTP Strict Transport Security):** This header tells the browser to automatically upgrade all requests to HTTPS, preventing the initial unencrypted HTTP request that could be intercepted 21, 22.
- **Configure CORS (Cross-Origin Resource Sharing):** Explicitly whitelist which domains are allowed to make requests to your API to prevent malicious sites from accessing your data 23, 24.
- **Defend Against CSRF (Cross-Site Request Forgery):**
- Use **Anti-CSRF Tokens** that the attacker cannot guess 25, 26.
- Set the **SameSite** attribute on cookies to Lax or Strict to prevent them from being sent with cross-site requests 27, 28.

Backend Security: Protecting the Server and Database

- **Prevent SQL Injection:** Never use string concatenation to build database queries. Instead, use **Parameterized Queries** (prepared statements) that treat user input as data rather than code 29, 30.
- **Implement Rate Limiting:** Limit the number of requests a user or IP can make to prevent brute-force attacks on login forms and DOS (Denial of Service) attacks 31-33.
- **Secure Error Handling:** Log detailed errors internally for debugging, but always return **generic error messages** to the user. Never expose stack traces, database schema details, or internal IP addresses in production 34, 35.

Identity and Access Management

- **Verify Identity (AuthN) vs. Permissions (AuthZ):** Authentication proves who the user is, while Authorization determines what they can do once logged in 36, 37.
- **Secure Password Hashing:** Never store plaintext passwords or use reversible encryption. Use slow, memory-hard, one-way hashing algorithms like **bcrypt** or **Argon2** combined with a unique **salt** for every user to prevent rainbow table attacks 38-41.
- **Manage JWTs Securely:**
 - Always verify the **expiration claim (exp)** and the **signature** 42, 43.
 - Explicitly whitelist allowed algorithms and **never accept alg: none** 44, 45.
 - Use cryptographically strong, random secret keys stored in environment variables 44, 45.
- **Implement Role-Based Access Control (RBAC):** Use server-side middleware to verify a user's role before granting access to specific API endpoints or resources 6, 41, 46.